

HybriChain: 基于 eBPF 的开源 AI 推理引擎的跨语言调用链追踪方法

本科毕业论文

四川大学网络空间安全学院

2026 年 6 月

汇报结构

- ① 研究背景与动机
- ② 相关工作及其不足
- ③ 整体方法与系统设计
- ④ Ollama 特殊工程问题及解决方案
- ⑤ 实验设计与结果分析
- ⑥ 本文创新点
- ⑦ 总结与展望

研究背景：AI 推理引擎的复杂性危机

开源大模型私有化部署爆发

- Ollama、llama.cpp 等开源推理框架成为企业/个人部署 LLM 的主流选择。
- 为满足极致性能，底层架构采用多语言混合编程与多进程解耦。

安全脆弱性暴露

- **CVE-2023-43654** (TorchServe): API 未授权上传 → 反序列化 RCE。
- **CVE-2024-37032** (Ollama): 畸形 API 数据 → 目录遍历越权读取。

核心矛盾：推理引擎内部数据流转对安全运维人员几乎完全不透明，现有观测工具无法连贯还原从 API 接收到内核调用的完整链路。

相关工作及其不足

- **Ollama 及同类推理引擎**

- 多语言混合编程 + 多进程解耦架构，导致内部数据流复杂且不透明。
- 缺乏内建的可观测性接口，调用链追踪依赖外部工具。

- **eBPF 动态追踪技术**

- 内核级零侵入观测，支持 uprobe/tracepoint 多锚点挂载。
- 现有应用多集中于网络/存储领域，尚未针对多语言 AI 推理引擎形成系统性方案。

- **现有调用链追踪方案**

- strace/ftrace：仅系统调用层，无法观测用户态函数级行为。
- 动态二进制插桩 (Valgrind/DynamoRIO)：性能退化 10-100 倍，不可用于生产环境。
- APM 工具 (Jaeger/SkyWalking)：需代码侵入插桩，对跨语言边界支持薄弱。

共性不足：控制流在跨语言边界时完全断裂，且各语言符号空间割裂，无法建立统一语义关联。

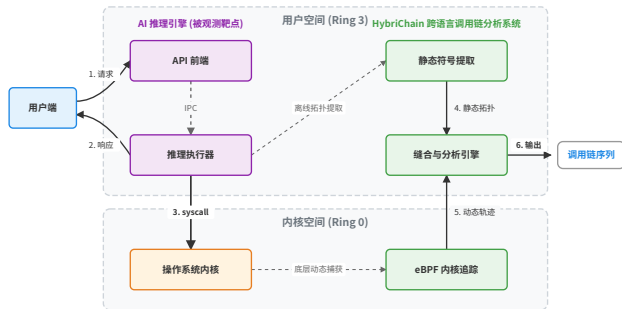
研究目标与方法概述

设计一套动静结合、低开销、通用的底层追踪框架，适用于所有采用多语言混合架构的 AI 推理引擎：

- ① **跨语言符号映射**。将来自不同语言命名空间的函数符号统一映射到同一语义空间。
- ② **eBPF 内核级追踪**。利用 eBPF 在零侵入前提下，对推理引擎进行函数与系统调用的双层观测。
- ③ **统一调用链缝合**。将用户态与内核态事件按时间戳缝合为端到端推理调用链，并赋予置信状态。
- ④ **调用链下游分析**。基于端到端调用链图谱，支持异常行为检测、性能瓶颈定位、安全审计与容量规划等多种分析场景。

本论文以 Ollama 为实验靶点验证方法有效性，但方法本身不依赖特定引擎实现，可推广至 llama.cpp、vLLM 等多语言推理框架。

系统总体架构



HybriChain 由三大核心模块构成：

- **静态符号提取模块**：离线解析推理引擎二进制文件，提取各层函数符号。
- **eBPF 内核追踪模块**：实时捕获系统调用与用户态函数调用事件流。
- **缝合与分析引擎**：将静态拓扑与动态轨迹融合，输出带置信状态的调用链图谱。

eBPF 探针下沉策略

推理引擎每秒产生数万次调用事件，全量追踪将导致存储和计算资源迅速耗尽。

HybriChain 采用探针下沉策略，将追踪点精确挂载至三个关键层次：

- ① **推理计算函数层 (uprobe)**：在张量计算核心函数上挂载，获取微秒级耗时数据。
- ② **系统调用层 (tracepoint)**：追踪 read/write/futex/mmap/clone3 等与推理语义相关的系统调用。
- ③ **内存分配层 (mmap tracepoint)**：专门追踪大块内存分配，分析 KV Cache 缓存行为。

eBPF 相比 strace/DTrace 的优势：内核运行、零侵入、热更新、RingBuf 零拷贝传输。

漏斗式降噪流水线

原始内核事件流每秒数万条，其中大量是与推理无关的噪声。HybriChain 设计五步漏斗降噪：

- **步骤一：元信息过滤。**过滤追踪工具启动/停止提示行。
- **步骤二：进程过滤。**仅保留推理引擎相关进程的事件。
- **步骤三：类型过滤。**白名单保留 read/write/futex/mmap，过滤 brk/mprotect/madvise 等噪声。
- **步骤四：参数过滤。**read 仅记录有效 fd，mmap 仅记录 $\geq 1\text{MB}$ 的分配。
- **步骤五：栈帧清洗。**过滤内存地址帧，保留可读符号名并 demangle。

置信度缝合算法

降噪后的数据是两类分散事件 (uprobe + tracepoint)，处于不同时间精度 (微秒 vs 毫秒)，无法直接拼接。

HybriChain 基于时间戳对齐的启发式缝合算法，为每条跨语言边赋予置信状态：

- **Confirmed (确认)**：该边两端节点在 ≥ 2 棵不同请求树中共同出现。
- **Inferred (推断)**：仅在 1 棵请求树中共同出现，可能存在时间窗口误匹配。
- **Unknown (未知)**：静态存在但动态未观测到，调用路径未被触发。

缝合输出：按时间排序的统一调用序列，每条事件携带置信状态与风险等级。

五层符号分类体系

缝合后的调用链中每条边对应一个函数符号，但这些符号来自完全不同的命名空间。

层次	名称	说明
f_0	系统边界层	POSIX 标准函数 (open/read/write/mmap), 与语言无关
f_1	运行时胶水层	语言绑定函数 (如 cgo 桥接、Python C Extension 等), 跨语言调用关键节点
f_2	张量计算层	矩阵乘法、注意力机制等核心计算 (matmul、attention)
f_3	内存管理层	缓存分配、模型权重加载、KV Cache 管理
f_4	调度协调层	任务分发、线程池管理、futex 同步操作

跨层调用举例: $f_1 \rightarrow f_2$ (通过语言桥接调用底层计算)、 $f_2 \rightarrow f_0$ (推理计算触发 mmap 分配)。

Ollama 特殊工程问题及解决方案

以 Ollama 为例，验证方法有效性。需解决以下多语言混合架构特有的工程问题：

语言绑定函数命名不确定

问题：多语言混合架构中，语言绑定层（如 CGO）编译器生成的函数符号含哈希后缀（如 `_cgoexp_HASH_`），无法直接 demangle 与标准化分类。

解决：通过识别语言绑定函数的前缀特征（`_cgo_`、`_Py_` 等），提取原始函数名，将其正确分类至胶水层（`f1`）。

多进程通信隔离

问题：推理引擎 daemon 进程与计算 runner 进程通过 Unix Domain Socket 通信，控制流跨越进程边界，无法在单进程追踪中缝合。

解决：在 daemon 与 runner 双进程上同时挂载 uprobe，通过请求 ID 关联跨进程事件，建立完整调用链。

实验设计

● 实验环境

- CPU-only 环境，评估 eBPF 探针在纯软件推理场景下的追踪能力。
- 测试模型：TinyLlama-1.1B、Qwen2-0.5B。

● 评估维度

- 调用链缝合准确性：基于 20 条 Ground Truth 基准路径（CGO 桥接、采样、调度、模型加载、批量处理链路）。
- 系统性能开销：推理时延（TTFT）、内存占用、数据处理时间。
- 异常行为检测能力：在真实采样场景下检测 `futex` 阻塞、异常内存分配、敏感路径访问等。

● 对照组

- 无 eBPF 基线、`strace` 全量追踪对比。

实验结果：调用链缝合准确性

基于 20 条 Ground Truth 基准路径验证：

指标	数值	说明
精确率	全部确认	零假阳性，缝合确认的每一条边均真实可靠
召回率	5%	仅直接观测确认 3/20 条 CGO 桥接边
含推断召回率	40%	加入 Inferred 边后提升至 12/20
Confirmed 边	3	CGO 桥接链路与调度链路
Inferred 边	11	时间窗口启发式推断

召回率低的原因：仅在 4 个核心函数上挂载 uprobe，探针覆盖不完整。3 条 Confirmed 边恰好是最有价值的跨语言边界边。

实验结果：系统性能开销

推理时延 (TTFT)

- TinyLlama: 65ms $\rightarrow$ 66ms (+1.5%)
- Qwen2: 64ms $\rightarrow$ 68ms (+6.3%)

eBPF 追踪对推理时延的影响可忽略不计。

降噪压缩效果

- TinyLlama: 25,329 $\rightarrow$ 8,891 事件 (压缩至 35%)
- Qwen2: 13,670 $\rightarrow$ 6,373 事件 (压缩至 47%)

有效过滤无关噪声，大幅降低下游分析压力。

内存占用

- bpftrace 进程 RSS 约 4MB
- 相对推理引擎数 GB 内存占用可忽略

追踪进程资源

eBPF 程序运行于内核空间，无需在用户态额外占用推理进程的 CPU 和内存资源。

结论： HybriChain 在几乎不影响推理引擎性能的前提下完成跨语言调用链追踪。

异常行为发现

基于调用链图谱实施规则驱动异常检测：

异常检测结果

在 Qwen2-0.5B 模型 23.83 秒采样中，发现 30 条 HIGH 级别 `futex` 阻塞告警：

- 最大单次阻塞 **15.2 秒** (TID=45633)
- 并发峰值：9 条告警同时触发 (9-10 秒区间)
- 揭示了 CPU-only 环境下多线程同步的结构性瓶颈与 DoS 脆弱点

此外还检测到异常内存分配 (`mmap_size`) 与敏感路径访问 (`sensitive_path`) 事件，验证了调用链下游分析在多安全场景中的适用性。

本文创新点

与现有跨语言追踪方案相比，HybriChain 的创新体现在：

创新 1：eBPF + 静态符号表的动静结合方案

首次将 eBPF 内核级动态追踪与静态二进制符号分析结合用于 AI 推理引擎，克服了纯静态方案（无法获取运行时行为）和纯动态方案（无法还原完整调用图）的各自局限。

创新 2：五层符号分类体系 (f_0 - f_4)

针对多语言混合架构中多种命名空间割裂的问题，提出统一的五层语义分类模型。更进一步，针对语言绑定层符号命名不确定的难题，设计了基于前缀特征的命名还原技术，将编译器生成的不可读符号（如含哈希后缀的语言绑定函数）还原为原始函数名，从而实现精确的跨语言符号分类。该技术不绑定特定引擎实现，适用于所有多语言混合推理框架。

创新 3：时间戳对齐的置信度缝合算法

将微秒精度（uprobe）与毫秒精度（tracepoint）异构事件流按时间窗对齐，自动判定 Confirmed/Inferred/Unknown 三种置信状态，解决了异构数据源无法直接拼接的难题。

研究局限与未来展望

当前局限性：

- **未覆盖 GPU 推理路径。** 当前实验在 CPU-only 环境下进行，未涉及 CUDA kernel 调用与 GPU 内存分配的追踪。
- **召回率受探针覆盖限制。** Confirmed 召回率仅 10%，源于 uprobe 仅挂载 4 个核心函数，大量子函数未被覆盖。
- **23 秒采样为 PoC 切片。** 长尾调用路径在采样窗口内未被触发，需延长采样时间。
- **异常检测依赖人工规则。** 对未知异常类型缺乏自适应学习能力。

未来工作方向：

- **GPU 路径覆盖。** 引入 CUPTI 追踪 CUDA API，建立 Python → CUDA kernel 完整调用链。
- **提升探针覆盖率。** 增加 uprobe 目标函数数量，扩展采样窗口为生产级基准。
- **智能化异常检测。** 引入自编码器 / LSTM 时序异常检测，减少对人工规则的依赖。

谢谢！

感谢各位老师，请批评指正。